

**Table 1-1 Multithreading Terms**

| <b>Term</b>                                    | <b>Definition</b>                                                                                                                                                                                                                                                                                          |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Process</b>                                 | The UNIX environment, such as file descriptors, user ID, and so on, created with the fork(2) system call, which is set up to run a program.                                                                                                                                                                |
| <b>Thread</b>                                  | A sequence of instructions executed within the context of a process.                                                                                                                                                                                                                                       |
| <b>POSIX pthreads</b>                          | A threads interface that is POSIX threads compliant. See <a href="#">Oracle Solaris Multithreading Libraries and Standards</a> for more information.                                                                                                                                                       |
| <b>Oracle Solaris threads</b>                  | An Oracle Solaris threads interface that is not POSIX threads compliant. A predecessor of pthreads.                                                                                                                                                                                                        |
| <b>Single-threaded</b>                         | Restricts access to a single thread. Execution is through sequential processing, limited to one thread of control.                                                                                                                                                                                         |
| <b>Multithreading</b>                          | Allows access to two or more threads. Execution occurs in more than one thread of control, using parallel or concurrent processing.                                                                                                                                                                        |
| <b>User-level or Application-level threads</b> | Threads managed by threads routines in user space, as opposed to kernel space. The POSIX pthreads and Oracle Solaris threads APIs are used to create and handle user threads. In this manual, and in general, a thread is a user-level thread.                                                             |
| <b>Lightweight processes</b>                   | Kernel threads, also called LWPs, that execute kernel code and system calls. LWPs are managed by the system thread scheduler, and cannot be directly controlled by the application programmer. Beginning with Solaris 9, every user-level thread has a dedicated LWP. This is known as a 1:1 thread model. |
| <b>Bound thread (obsolete term)</b>            | Prior to Solaris 9, a user-level thread that is permanently bound to one LWP. Beginning with Solaris 9, every thread has a dedicated LWP, so all threads are bound threads. The concept of an unbound thread no longer exists.                                                                             |
| <b>Unbound thread (obsolete term)</b>          | Prior to Solaris 9, a user-level thread that is not necessarily bound to one LWP. Beginning with Solaris 9, every thread has a dedicated LWP, so the concept of unbound threads no longer exists.                                                                                                          |
| <b>Attribute object</b>                        | Contains opaque data types and related manipulation functions. These data types and functions standardize some of the configurable aspects of POSIX threads, mutual exclusion locks (mutexes), and condition variables.                                                                                    |
| <b>Mutual exclusion locks</b>                  | Objects used to lock and unlock access to shared data. Such objects are also known as mutexes.                                                                                                                                                                                                             |
| <b>Condition variables</b>                     | Objects used to block threads until a change of state.                                                                                                                                                                                                                                                     |
| <b>Read-write locks</b>                        | Objects used to allow multiple read-only access to shared data, but exclusive access for modification of that data.                                                                                                                                                                                        |
| <b>Counting semaphore</b>                      | A memory-based synchronization mechanism in which a non-negative integer count is used to coordinate access by multiple threads to shared resources.                                                                                                                                                       |
| <b>Parallelism</b>                             | A condition that arises when at least two threads are executing                                                                                                                                                                                                                                            |

simultaneously.

**Concurrency**      A condition that exists when at least two threads are making progress. A more generalized form of parallelism that can include time-slicing as a form of virtual parallelism.

## **1. Introduction to Multithreading in Sorting**

Sorting can be a time-consuming operation when dealing with large datasets. By employing multithreading, we can split the dataset into smaller chunks and sort these chunks in parallel, thereby potentially reducing the overall execution time.

## **2. Merge Sort with Multithreading**

**Merge Sort** is a popular sorting algorithm that employs a divide-and-conquer approach. It divides the array into halves, recursively sorts them, and then merges the sorted halves. This makes it a good candidate for multithreading.

### **Example: Multithreaded Merge Sort**

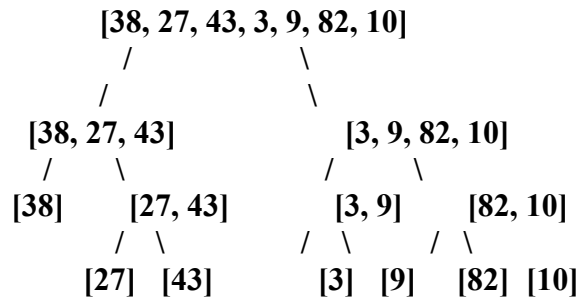
#### **1. Initial Array**

Let's say we have an unsorted array:

**Initial Array:** [38, 27, 43, 3, 9, 82, 10]

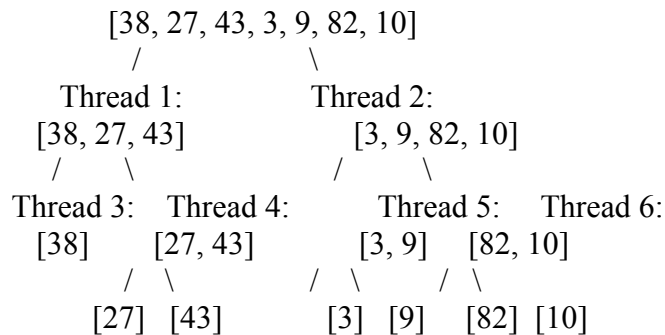
## 2. Recursive Division

When we start the process by calling `mergeSort`, the array will be divided into halves recursively. Here is how the array splits:



## 3. Thread Creation

At each level of recursion, separate threads can be created to sort the left and right halves:



- Thread 1 for sorting [38, 27, 43]
- Thread 2 for sorting [3, 9, 82, 10]
- Thread 3 for [38] (single element, base case)
- Thread 4 for sorting [27, 43]
- Thread 5 for sorting [3, 9]
- Thread 6 for sorting [82, 10]

## 4. Merging

Once the threads have completed sorting their respective parts, they will merge the results:

Merging [27] and [43]: [27, 43]

The left side merges to become: [27, 38, 43]

(Note that while merging, Thread 1 will have acquired results from Thread 3 and Thread 4).

For the right side:

Merging [3] and [9] gives: [3, 9]

Merging [82] and [10] results in: [10, 82] (note the sorting)

Now, merging [3, 9] and [10, 82] gives: [3, 9, 10, 82]

Final Merge:

Now, both halves are merged: [27, 38, 43] and [3, 9, 10, 82]

Final Sorted: [3, 9, 10, 27, 38, 43, 82]

This diagram outlines how the multithreaded approach to merge sort decomposes the problem into smaller, independent parts, which can be solved concurrently. The effectiveness of using multithreading here lies in the ability to utilize multiple processors or cores to handle separate subarrays at the same time, significantly improving the sorting time for large datasets.

Here is a simple example illustrating how to implement multithreading with the Merge Sort algorithm in Java.

```
import java.util.Arrays;

class MergeSortThread extends Thread {

    private int[] arr;

    private int left;

    private int right;

    public MergeSortThread(int[] arr, int left, int right) {

        this.arr = arr;

        this.left = left;

        this.right = right;

    }

    @Override

    public void run() {

        if (left < right) {

            int mid = (left + right) / 2;

            // Create two threads for the two halves of the array

            MergeSortThread leftThread = new MergeSortThread(arr, left, mid);

            MergeSortThread rightThread = new MergeSortThread(arr, mid + 1, right);

            leftThread.start();

            rightThread.start();

            try {

                // Wait for both threads to complete

                leftThread.join();

                rightThread.join();

            } catch (InterruptedException e) {
```

```

        e.printStackTrace();
    }

    // Merge the sorted halves
    merge(arr, left, mid, right);
}
}

private void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int[] L = new int[n1];
    int[] R = new int[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    // Merge the temporary arrays
    int i = 0, j = 0;
    int k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
}

```

```

    }

    // Copy remaining elements of L[] if any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy remaining elements of R[] if any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

}

public class Main {
    public static void main(String[] args) {
        int[] arr = {38, 27, 43, 3, 9, 82, 10};

        System.out.println("Unsorted array: " + Arrays.toString(arr));

        // Create a thread for the entire sorting operation
        MergeSortThread sortThread = new MergeSortThread(arr, 0, arr.length - 1);
        sortThread.start();

        try {
            sortThread.join(); // Wait for the sorting to complete
        } catch (InterruptedException e) {

```

```

        e.printStackTrace();
    }

    System.out.println("Sorted array: " + Arrays.toString(arr));
}
}

```

### Advantages of Multithreading in Sorting

Increased Performance: Sorting large arrays can be expedited by splitting the work across multiple threads. Ex. DBMS(querying, sorting, indexing)

When you execute a query that retrieves a sorted list of user records based on a certain criterion (e.g., sorting customer records by their names or purchase amounts), the database management system might use multithreaded sorting techniques like **Timsort** (used in Python and Java's Collections framework) or **parallel merge sort** to speed up the processing of large tables.

### Big Data Processing Frameworks

Frameworks like Apache Hadoop and Apache Spark are designed to handle large-scale data processing. They sort large datasets efficiently by distributing the workload across multiple nodes.

### File Systems and Data Storage

Operating systems and file systems implement sorting algorithms for file management (e.g., for directory listings or managing file metadata).

- **Example:** When retrieving a sorted list of files in a directory, the file system may utilize multithreaded sorting methods to sort a large number of files efficiently, particularly with file systems managing large datasets like repositories for data science.

### Real-time Data Processing

Applications that handle streaming data often need to sort incoming data for analytics, processing events, or transformations.

- **Example:** A stock market application might receive thousands of transactions per second. It can use a multithreaded version of a sorting algorithm to maintain a sorted list of transactions by timestamp or price to provide real-time analytics.

### Search Engines

Search engines index vast amounts of web data and require sorting algorithms to arrange data efficiently for quick retrieval and ranking.

- **Example:** A search engine may index pages based on keywords or relevance scores. When generating search results, a multithreaded sorting algorithm helps to process and organize



this indexed data for fast access. The page ranking process often involves sorting the results based on various criteria, such as PageRank or other metrics.

### **Machine Learning and Data Analysis**

In machine learning and data analysis, sorting is often used for preparing datasets and feature selection.

- **Example:** When preprocessing datasets that contain a large number of samples, a multithreaded sorting algorithm might be applied to ensure features are organized for model training. Libraries like TensorFlow and Scikit-learn leverage efficient sorting algorithms within preprocessing pipelines.

**Concurrent Execution:** Multiple sections can be sorted simultaneously, leading to a faster overall sort.

**Improved Resource Utilization:** Makes better use of CPU resources, especially in a multi-core environment.

### **4. Considerations/Disadvantages**

**Overhead:** Thread management introduces overhead. For small datasets, multithreading may not yield performance gains.

**Complexity:** Synchronization implementation might be needed if multiple threads access shared data structures.

**Error Handling:** Make sure to handle interruptions and exceptions gracefully to avoid application crashes.

## Lifecycle of a thread

In Java, the lifecycle of a thread is defined by a series of states, representing the various phases a thread can be in during its execution. Understanding the thread lifecycle is crucial for writing efficient multithreaded applications. Here's an overview of the thread lifecycle, including the various states and transitions between them.

### Thread Lifecycle States

#### 1. New:

- When a thread is created but not yet started, it is in the new state. You create a thread by instantiating the Thread class.

- **Example:**

```
Thread thread = new Thread();
```

#### **Runnable:**

- A thread enters the runnable state when the start() method is invoked. The thread is ready to run and is waiting for CPU time.
- A thread can be in this state for various reasons: it may be actively running, or it could be waiting for CPU allocation.

- **Example:**

```
thread.start();
```

#### **Blocked:**

- A thread enters the blocked state when it is waiting for a monitor lock (used for synchronization) to enter a synchronized block or method. In this state, the thread is unable to proceed until it acquires the lock.

- **Example:**

```
synchronized (someObject) {  
    // some code  
}
```

#### **Waiting:**

- A thread enters the waiting state when it is waiting for another thread to perform a specific action (like notifying or interrupting). This can occur when a thread calls methods such as Object.wait(), Thread.join(), or LockSupport.park().

- **Example:**

```
synchronized (someObject) {  
    someObject.wait(); // thread will wait until notified  
}
```

### **Timed Waiting:**

- A thread enters this state when it is waiting for another thread to perform an action for a specified waiting time. Methods such as `Thread.sleep(millis)`, `Object.wait(millis)`, or `Thread.join(millis)` can lead to this state.

- **Example:**

`Thread.sleep(1000);` // thread will sleep for 1 second

### **Terminated:**

- A thread enters the terminated state when it has completed its execution, either successfully or due to an exception. Once a thread is terminated, it cannot be restarted.
- This state can occur after the `run()` method has completed its execution (either by returning or throwing an exception).
- **Examples:**
  - If the thread completes normally:

```
public void run() {  
    // Thread execution code  
}
```

If an unhandled exception occurs during execution, the thread also enters a terminated state.

### **State Transition**

The transitions between these states can be summarized as follows:

- **From New to Runnable:**
  - By calling `start()`, a thread transitions from the New state to the Runnable state.
- **From Runnable to Running:**
  - The thread scheduler picks a thread in the Runnable state to run (this happens automatically; it cannot be explicitly controlled).
- **From Running to Blocked:**
  - When a thread tries to enter a synchronized block/method and the required lock is unavailable, it transitions to the Blocked state.
- **From Running to Waiting:**
  - A thread can become Waiting if it calls `wait()`, `join()`, or similar methods.
- **From Running to Timed Waiting:**
  - This occurs when a thread calls `sleep`, or `wait` with a timeout, or `join` with a timeout.
- **From Blocked to Runnable:**

- When the required lock becomes available, the thread can transition back to the Runnable state.
- **From Waiting to Runnable:**
  - A waiting thread becomes Runnable when another thread notifies it (with notify(), notifyAll(), etc.) or when it is interrupted.
- **From Running to Terminated:**
  - A thread enters the Terminated state when its run() method completes (either naturally or due to an exception).

```
+-----+
```

```
|      |
```

```
|  New  |
```

```
|      |
```

```
+-----+
```

```
  |
```

```
  |
```

```
  start()
```

```
  |
```

```
+-----+
```

```
|      |
```

```
| Runnable |
```

```
|      |
```

```
+-----+
```

```
/  |  \
```

```
/  |  \
```

```
running | Blocked
```

```
\  |
```

```
\  |
```

```
Waiting |
```

```
\  |
```

```
Timed- |
```

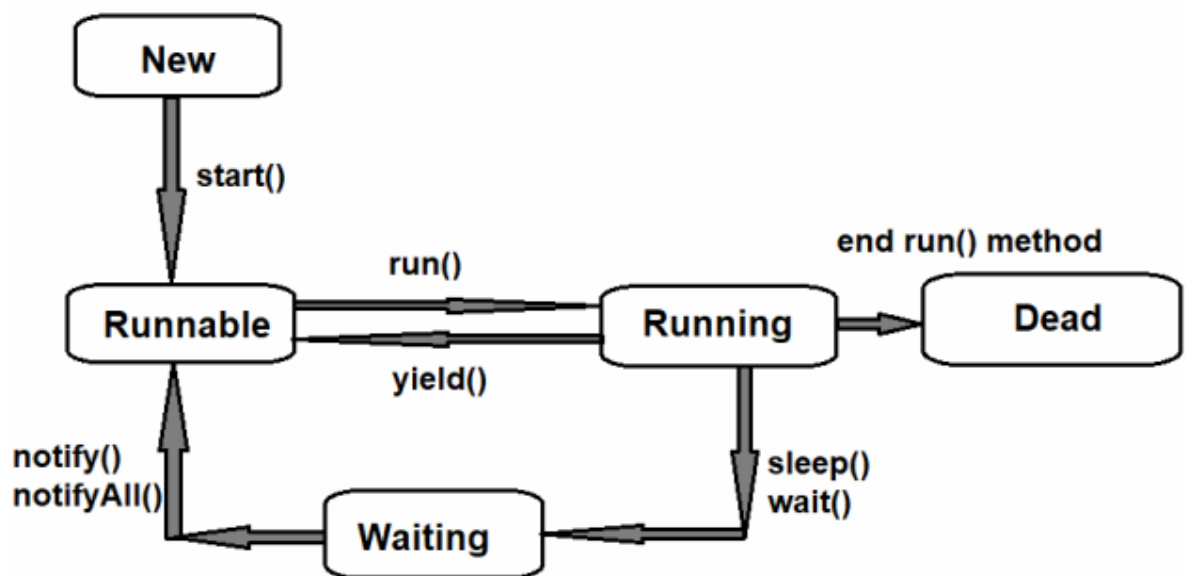
```
Waiting |
```

```
\  /
```

```

\ /
+-----+
|       |
| Terminated |
|       |

```



## Life Cycle of Thread in Java

1. New- When a thread is created but not yet started, it is in the new state. You create a thread by instantiating the `Thread` class.

Example:

```
Thread thread = new Thread();
```

2. Runnable - A thread enters the runnable state when the `start()` method is invoked. The thread is ready to run and is waiting for CPU time. A thread can be in this state for various reasons: it may be actively running, or it could be waiting for CPU allocation.

Example :

```
thread.start();
```

3. Blocked- A thread enters the blocked state when it is waiting for a monitor lock (used for synchronization) to enter a synchronized block or method. In this state, the thread is unable to proceed until it acquires the lock.

Example:

```
synchronized (someObject) {
    // some code }

```

4. Waiting- A thread enters the waiting state when it is waiting for another thread to perform a specific action (like notifying or interrupting). This can occur when a thread calls methods such as `Object.wait()`, `Thread.join()`, or `LockSupport.park()`.

Example:

```
synchronized (someObject) {  
    someObject.wait(); // thread will wait until notified  
}
```

5. Timed Waiting- A thread enters this state when it is waiting for another thread to perform an action for a specified waiting time. Methods such as `Thread.sleep(millis)`, `Object.wait(millis)`, or `Thread.join(millis)` can lead to this state.

Example:java

```
Thread.sleep(1000); // thread will sleep for 1 second
```

6. Terminated- A thread enters the terminated state when it has completed its execution, either successfully or due to an exception. Once a thread is terminated, it cannot be restarted. This state can occur after the `run()` method has completed its execution (either by returning or throwing an exception).

Examples:- If the thread completes normally

```
public void run() {  
    // Thread execution code  
}
```

If an unhandled exception occurs during execution, the thread also enters a terminated state.

The transitions between these states can be summarized as follows:

From New to Runnable - By calling `start()`, a thread transitions from the New state to the Runnable state.

From Runnable to Running - The thread scheduler picks a thread in the Runnable state to run (this happens automatically; it cannot be explicitly controlled).

From Running to Blocked - When a thread tries to enter a synchronized block/method and the required lock is unavailable, it transitions to the Blocked state.

From Running to Waiting- A thread can become Waiting if it calls `wait()`, `join()`, or similar methods.

From Running to Timed Waiting - This occurs when a thread calls `sleep`, or `wait` with a timeout, or `join` with a timeout.

From Blocked to Runnable - When the required lock becomes available, the thread can transition back to the Runnable state.

From Waiting to Runnable- A waiting thread becomes Runnable when another thread notifies it (with `notify()`, `notifyAll()`, etc.) or when it is interrupted.

From Running to Terminated - A thread enters the Terminated state when its `run()` method completes (either naturally or due to an exception).

### **Why Lifecycle for a thread in java?**

In Java, threads have a lifecycle to effectively manage their execution and resource allocations, which is crucial for ensuring that multithreaded applications run efficiently and without issues such as deadlocks, resource contention, and starvation. A lifecycle provides a structured framework that describes the various states a thread can be in from its creation to its completion. Here are the main reasons why a thread in Java has a lifecycle:

#### **1. State Management**

The lifecycle defines clear states for a thread (such as New, Runnable, Blocked, Waiting, Timed Waiting, and Terminated) which helps in managing its execution. Each state provides insight into what a thread is currently doing and allows developers to understand how their threads are behaving within the overall application.

Example: Knowing whether a thread is currently running, waiting for an I/O operation, or is terminated helps in debugging and performance tuning.

#### **2. Resource Optimization**

Using a lifecycle helps in optimizing system resource usage. Threads consume CPU cycles and memory. By managing their states efficiently, the Java runtime can better allocate resources to threads that are active while putting idle threads on hold, thus maximizing CPU utilization and improving overall application performance.

Example: Threads in the Runnable state are ready to execute but may not be running at every moment due to CPU scheduling. The lifecycle allows the system to manage this during execution.

#### **3. Control Over Execution**

The lifecycle provides mechanisms to control the execution flow of threads. Developers can create threads, start them, wait for them to complete, and ensure they terminate correctly. The lifecycle defines how and when these operations can happen.

Example: You can use methods like `Thread.sleep()`, `Object.wait()`, or `Thread.join()` to manage how threads enter the Waiting or Timed Waiting states, giving you control over the execution sequence.

#### **4. Synchronization and Concurrency Management**

During multithreaded operations, threads may need to communicate or share resources. The lifecycle allows for better synchronization practices and concurrency control by defining when a thread can enter the Blocked or Waiting states.

Example: When a thread tries to enter a synchronized block, it may get Blocked if another thread holds the lock, enabling proper synchronization of shared data.

#### **5. Error Handling and Cleanup**

The lifecycle also pertains to how threads handle errors and perform cleanup operations when they terminate. It provides a clear structure for developers to manage exceptions and to execute necessary cleanup code once a thread's task is complete.

Example: The `run()` method of a thread can throw exceptions. Once the exceptions are handled, the thread transitions to the Terminated state, allowing for proper resource release.

## 6. Debugging and Monitoring

Having a defined lifecycle for threads simplifies the debugging and monitoring of multithreaded applications. Developers can observe a thread's state to troubleshoot performance issues or deadlocks based on how threads transition from one state to another.

Example: Monitoring tools and profilers can check the state of threads and determine if they are stuck in a Blocked or Waiting state, which can indicate potential issues in the application.

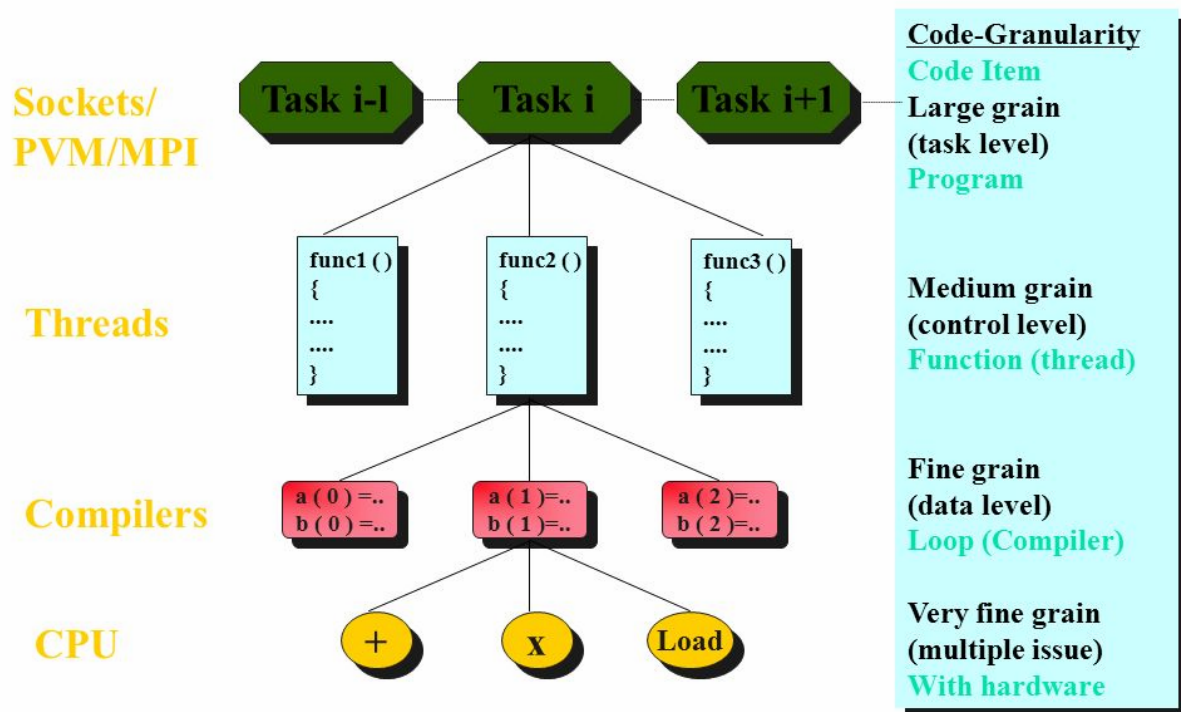
## Summary

The lifecycle of a thread in Java serves several critical functions that facilitate resource management, control over execution flow, synchronization, error handling, and monitoring. This structure enables developers to create robust, efficient, and maintainable multithreaded applications. By understanding the thread lifecycle, developers can better design their applications to leverage concurrency while avoiding common pitfalls associated with multithreading.



In Java, levels of parallelism refer to the various ways in which tasks can execute concurrently, utilizing multiple threads or processing units. Understanding these levels can help you design and implement efficient applications that take full advantage of multicore processors. Here are the main levels of parallelism in Java:

## Levels of Parallelism



### 1. Instruction-level Parallelism (ILP)

This is the lowest level of parallelism and is primarily handled by the CPU architecture itself. ILP allows a single processor core to execute multiple instructions in parallel by utilizing techniques such as pipelining and out-of-order execution.

Example: Modern CPUs can execute several instructions simultaneously within a single core. This happens at the level of individual instructions and does not require any changes from the Java code.

### 2. Thread-Level Parallelism (TLP)

This level of parallelism is achieved through the use of threads. TLP allows multiple threads to run on different CPU cores or interleave execution on a single core. In Java, this is accomplished through the `Thread` class or the `Runnable` interface and the Executor framework introduced in Java 5.

Example: You can create multiple threads that perform different tasks at the same time. For instance, calculating different parts of a large dataset concurrently.

```
class MyTask implements Runnable {
```

```

    public void run() {
        // Task implementation here
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyTask());
        Thread t2 = new Thread(new MyTask());

        t1.start();
        t2.start();
    }
}

```

### 3. Process-Level Parallelism

This level involves running multiple processes in parallel. Each process has its own memory space and runs independently. In Java, this can be achieved through the Java Process API or by using tools like Java Fork/Join framework for handling parallel processing across multiple processes.

Example: Java applications can utilize the `ProcessBuilder` class to start new processes, perhaps for executing external commands or applications.

```

ProcessBuilder processBuilder = new ProcessBuilder("externalCommand");
Process process = processBuilder.start();

```

### 4. Data-Level Parallelism

This level of parallelism focuses on distributing data among multiple threads and performing the same operation on different pieces of data simultaneously. In Java, this can be achieved through parallel streams or libraries such as Fork/Join.

Example: Using parallel streams introduced in Java 8, you can leverage data-level parallelism with minimal effort.

```

List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> squares = numbers.parallelStream()
    .map(n -> n * n)
    .collect(Collectors.toList());

```

### 5. Task-Level Parallelism

This level refers to executing different tasks at the same time, where each task may perform a different kind of operation. Task-level parallelism can be easily implemented using the Java Executor framework, which simplifies the creation and management of threads.

Example: Using `ExecutorService` to submit tasks for concurrent execution.

```
ExecutorService executor = Executors.newFixedThreadPool(3);
```

```
executor.submit(() -> {  
    // Task 1 implementation
```

```
});
```

```
executor.submit(() -> {  
    // Task 2 implementation
```

```
});
```

```
executor.shutdown();
```

## 6. Pipeline parallelism

This form of parallelism involves breaking down a task into multiple stages, with each stage being processed in parallel. It's often used in scenarios where a task can be divided into distinct phases that can execute concurrently.

Example: In a data processing application, one stage might read data, a second stage processes it, and a third stage outputs the results. Each stage can be handled by separate threads, with data being passed between them.

```
ExecutorService executor = Executors.newFixedThreadPool(3);
```

```
executor.submit(() -> {  
    // Stage 1: Data Reading
```

```
});
```

```
executor.submit(() -> {  
    // Stage 2: Data Processing
```

```
});
```

```
executor.submit(() -> {  
    // Stage 3: Data Output
```

```
});
```

## Summary

Understanding these levels of parallelism in Java is essential for developing efficient applications that maximize the use of available resources. Developers can leverage built-in libraries and language features to implement various levels of parallelism, enabling them to tackle more complex

computational problems. By utilizing the appropriate level of parallelism, applications can achieve improved performance and responsiveness, especially in computationally intensive scenarios.